Unsupervised Hierarchical Multi-Label Text Classification with Silver Labels and Self-Training

MINJUN KOO, Data Science, 2024320333

1 Introduction

Hierarchical Multi-Label Text Classification (HMTC) aims to assign each document to multiple categories organized in a hierarchical taxonomy. In this project, we address an *unsupervised* HMTC setting, where no human-annotated labels are available. Instead, we are given only an unlabeled corpus of product reviews, a predefined class taxonomy, and class-related keywords.

The absence of labeled data makes conventional supervised approaches inapplicable and introduces ambiguity among semantically similar classes. To tackle this challenge, we generate *silver labels* using semantic similarity between document embeddings and class representations, and refine them using confidence thresholds and hierarchy-aware constraints. These silver labels are then used to train a multi-label classifier, which is further improved through iterative self-training.

This framework enables effective learning without any manually annotated data while explicitly leveraging the hierarchical structure of the taxonomy.

2 Dataset and Task Setup

2.1 Unlabeled Reviews and Test Data

The dataset consists of an unlabeled training corpus of **29,487 product reviews** and a test set of **19,658 reviews** used for evaluation. Each review is a free-form text describing user experiences with a product. No ground-truth labels are provided for the training corpus, and all supervision must be derived automatically.

2.2 Class Taxonomy and Keywords

The classification space contains **531 product categories** organized as a directed acyclic graph (DAG), representing a hierarchical taxonomy. Each node corresponds to a product class, and edges represent parent–child relationships between more general and more specific categories. In addition to the taxonomy structure, each class is associated with a small set of class-related keywords, which provide weak semantic cues about the class meaning.

The hierarchical structure imposes consistency constraints on predictions: if a document is assigned to a child class, it should also be assigned to its ancestor classes. Our method explicitly considers this constraint during pseudo-label generation and prediction.

2.3 Evaluation Protocol

Model performance is evaluated through a Kaggle competition, where predictions are submitted as binary label vectors for each test document. Since ground-truth labels are not available for the training data, the Kaggle leaderboard score serves as the primary evaluation metric. All models are trained using the same unlabeled corpus and auxiliary information to ensure a fair comparison.

Author's Contact Information: Minjun Koo, Data Science, 2024320333, mjkoo1226@korea.ac.kr.

ACM XXXX-XXXX//12-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

3 Silver Label Generation

Silver label generation is the core component of our framework, as it provides the only supervision signal for training the model. Our method combines similarity-based candidate selection, confidence-based filtering, and hierarchy-aware label completion to generate reliable pseudo labels from unlabeled data.

3.1 Document Embeddings

Each document d_i is mapped to a fixed-dimensional vector representation

$$\mathbf{e}_i \in \mathbb{R}^D$$

using a **pretrained sentence embedding model**.

Specifically, we use the **all-mpnet-base-v2** model from the *Sentence-Transformers* library, which is based on the MPNet architecture and pretrained on large-scale sentence-level semantic similarity datasets. This model is well-suited for capturing fine-grained semantic relationships between short and medium-length texts, such as product reviews.

Formally, the embedding of a document is computed as:

$$\mathbf{e}_i = \text{MPNet}(d_i).$$

All document embeddings are L2-normalized before similarity computation to ensure stable cosine similarity scores across documents. The embedding model is kept **frozen during training** and used solely for feature extraction.

3.2 Similarity-based Label Assignment (Top-K)

For each product category, we construct a class representation by concatenating the class name with its associated keywords and encoding the resulting text using the same sentence embedding model as for documents. Given a document embedding, cosine similarity is computed against all class representations.

Due to the multi-label nature of the task and the large number of classes, we retain only the Top-K most similar classes for each document as candidates. This significantly reduces noise from low-confidence classes while preserving potentially relevant labels.

3.3 Thresholding and Fallback Strategy

Among the Top-K candidate classes, we apply a similarity threshold τ to filter unreliable assignments. Classes whose similarity scores exceed τ are selected as pseudo labels.

Strict thresholding may result in some documents receiving no labels. To guarantee full coverage of the training corpus, we apply a fallback strategy: if no candidate exceeds the threshold, the single most similar class is assigned.

3.4 Hierarchy-aware Label Completion

To ensure consistency with the class taxonomy, all ancestor classes of an assigned label are also included. This upward-only propagation enforces hierarchical validity while avoiding aggressive label expansion to unrelated child nodes.

To enforce consistency with the class taxonomy, we expand the selected pseudo labels by propagating them upward in the hierarchy. If a document d_j is assigned to a class c_j, then all ancestor classes of c_j are also assigned:

$$\tilde{y}_{ia} = 1, \quad \forall a \in \text{Ancestors}(c_j).$$

This hierarchy-aware completion aligns the pseudo labels with the directed acyclic graph structure of the taxonomy and prevents violations such as predicting a fine-grained class without its parent categories.

3.5 Pseudo-label Statistics

After silver label generation and hierarchy-aware completion, all documents are assigned at least one pseudo label, ensuring full coverage of the unlabeled training corpus. The average number of labels per document increases slightly from 5.37 to 5.39 after hierarchical propagation, corresponding to 460 additional labels across the entire dataset (1.48

This small increase indicates that hierarchy-aware completion is conservative and does not cause label explosion. The resulting silver labels preserve structural consistency while remaining primarily driven by high-confidence semantic similarity, providing a stable supervision signal for subsequent model training.

4 Training Process

4.1 Multi-Layer Perceptron (MLP) with Self-Training

As a baseline classifier, we employ a 3-layer Multi-Layer Perceptron (MLP) that takes 768-dimensional document embeddings as input.

4.1.1 Initial Supervised Training. Let

$$f_\theta(d_i) \in [0,1]^{|C|}$$

denote the predicted probability vector for document d_i.

We optimize the Binary Cross-Entropy loss over the generated silver labels:

$$\mathcal{L}_{\text{BCE}} = -\sum_{i,j} \Big[y_{ij} \log f_\theta(d_i)_j + (1 - y_{ij}) \log \big(1 - f_\theta(d_i)_j\big) \Big].$$

The model is trained using the Adam optimizer with a learning rate of 1×10^{-4}.

4.1.2 Self-Training via Iterative Pseudo-label Refinement. After initial convergence, we apply an iterative self-training procedure. At iteration t, the model generates pseudo-labels

$$\hat{y}_{ij}^{(t)}$$

for the training set.

Only predictions exceeding a confidence threshold γ are retained:

$$\hat{y}_{ij}^{(t)} = \mathbb{I}\big[f_\theta^{(t)}(d_i)_j \geq \gamma\big].$$

These high-confidence predictions are merged with the existing silver labels and used to retrain the model. This process leads to a consistent performance improvement, increasing the Kaggle score from **0.57036** to **0.57608**.

4.2 Single LabelGAT Layer: Transformer-style Multi-Head Attention on Graph

Each LabelGAT layer performs a Transformer-style attention update, but only over graph-permitted neighbors.

4.2.1 Q, K, V projections (per node). Given input

$$\mathbf{X} \in \mathbb{R}^{C \times d_{\text{in}}},$$

we compute query, key, and value matrices:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V,$$

where

$$\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}.$$

Here,

$$d_{\text{out}} = H \cdot d_h,$$

where H is the number of heads and d_h is the head dimension.

We reshape them into heads:

$$\mathbf{Q}_i, \mathbf{K}_j, \mathbf{V}_j \in \mathbb{R}^{H \times d_h}.$$

4.2.2 Scaled dot-product attention logits with graph masking. For each target node i and source node j, attention logits are computed per head:

$$e_{ij}^{(h)} = \frac{\langle \mathbf{Q}_i^{(h)}, \mathbf{K}_j^{(h)} \rangle}{\sqrt{d_h}}.$$

We then apply the taxonomy mask so that only allowed edges contribute:

$$e_{ij}^{(h)} = \begin{cases} e_{ij}^{(h)}, & \mathbf{A}_{ij} = 1, \\ -\infty, & \mathbf{A}_{ij} = 0. \end{cases}$$

This corresponds exactly to the code line that fills non-neighbor logits with `-inf` (implemented via `torch.finfo(...).min`).

4.2.3 Attention weights and aggregation. We normalize over the neighbor dimension:

$$\alpha_{ij}^{(h)} = \frac{\exp(e_{ij}^{(h)})}{\sum_{k:\mathbf{A}_{ik}=1} \exp(e_{ik}^{(h)})}.$$

Then aggregate values:

$$\mathbf{O}_i^{(h)} = \sum_{j:\mathbf{A}_{ij}=1} \alpha_{ij}^{(h)} \mathbf{V}_j^{(h)}.$$

Concatenating heads and applying an output projection:

$$\mathbf{O}_i = \text{Concat}\big(\mathbf{O}_i^{(1)}, \ldots, \mathbf{O}_i^{(H)}\big)\mathbf{W}_O,$$

where

$$\mathbf{W}_O \in \mathbb{R}^{d_{\text{out}} \times d_{\text{out}}}.$$

Dropout is applied on attention weights and on the projected output to regularize training.

4.2.4 Residual connection and Layer Normalization. Each LabelGAT layer uses a built-in residual connection (when dimensions match) and LayerNorm:

$$\mathbf{X}' = \text{LayerNorm}(\mathbf{O} + \mathbf{X}).$$

This stabilizes optimization and helps preserve the original semantic information contained in the initial class embeddings.

4.3 Multi-layer ClassOnlyGAT

We stack L LabelGAT layers:

$$\mathbf{X}^{(\ell+1)} = \mathrm{LabelGAT}^{(\ell)}\big(\mathbf{X}^{(\ell)}, \mathbf{A}\big), \quad \ell = 0, \ldots, L-1.$$

The final class prototypes are:

$$\mathbf{H}_c = \mathbf{X}^{(L)} \in \mathbb{R}^{C \times d}.$$

Intuitively, stacking layers allows class nodes to incorporate information from multi-hop taxonomy neighborhoods while still being constrained by the adjacency mask.

4.4 Prototype-based Document–Class Matching (GATPrototypeClassifier)

After obtaining class prototypes, we map each document embedding

$$\mathbf{z}_i \in \mathbb{R}^d$$

through a small MLP:

$$\mathbf{h}_i = \mathrm{MLP}(\mathbf{z}_i).$$

Both document features and class prototypes are ℓ_2-normalized:

$$\bar{\mathbf{h}}_i = \frac{\mathbf{h}_i}{\|\mathbf{h}_i\|}, \quad \bar{\mathbf{H}}_c = \frac{\mathbf{H}_c}{\|\mathbf{H}_c\|}.$$

We then compute cosine-similarity logits:

$$\mathrm{logit}_{ij} = \frac{\bar{\mathbf{h}}_i^\top \bar{\mathbf{H}}_{c_j}}{\tau_j}.$$

4.4.1 Class-wise temperature scaling. Instead of using a single global temperature, we use a class-wise temperature:

$$\tau_j = \exp(t_j),$$

where t_j is a learnable parameter (`log_temp[j]`). This allows the model to calibrate classes with different difficulty and frequency, improving multi-label decision boundaries under noisy supervision.

The final logits are passed to a sigmoid + BCE loss during training.

4.5 Graph Attention Network (GAT) with Momentum Stabilization

To incorporate taxonomic structure directly into the classifier, we employ a Graph Attention Network (GAT) over the class taxonomy.

4.5.1 Graph-based Class Representation. Each class node is initialized with a semantic embedding and updated via attention-based message passing over parent–child edges. However, due to the presence of noisy silver labels, naïve graph propagation often leads to over-smoothing and unstable training.

4.5.2 Momentum-based Parameter Update (EMA). Instead of skip or residual learning, we apply a momentum-based update mechanism, implemented as an Exponential Moving Average (EMA) over model parameters:

$$\theta_{\mathrm{EMA}}^{(t)} = \mu\, \theta_{\mathrm{EMA}}^{(t-1)} + (1-\mu)\, \theta^{(t)},$$

where

$$\mu \in [0, 1)$$

is the momentum coefficient, set to **0.999**.

Table 1. Kaggle performance of different model variants.

Model Variant	Kaggle Score
MLP Only	0.57036
MLP + Self-Training	**0.57608**
GAT Only	0.56502
GAT + Self-Training	0.56676
GAT (LLM) Only	0.53326
GAT (LLM) + Self-Training	0.53386
GAT + Momentum Only	0.56830
GAT + Momentum + Self-Training	0.56513

This mechanism acts as a temporal ensemble over training steps, smoothing parameter updates and mitigating divergence caused by noisy pseudo labels. Empirically, momentum stabilization consistently improves training stability compared to direct GAT optimization.

4.6 LLM-Enhanced Node Features

To explore whether richer semantic descriptions could improve class representations, we experiment with using a Large Language Model (LLM) to generate textual definitions for each class node in the taxonomy.

4.6.1 Prompting Strategy. For each product category, we prompt an LLM (**GPT-4o mini**) to generate a concise, neutral, and factual definition. The prompt template is as follows:

> You are given a product category name and its related keywords. Write a neutral and factual definition of this category in one sentence.
> *Category name:* c_j
> *Related keywords:* $\{k_1, k_2, \dots\}$
> The sentence should explain what this category refers to, without using marketing language or mentioning any specific brand or platform.

This design choice ensures consistency across all **531 classes** and limits stylistic variance that could introduce noise into downstream embedding models.

4.6.2 LLM-based Class Embedding Construction. Let T_j denote the generated definition text for class c_j. Each T_j is encoded using the same pretrained sentence embedding model used for document embeddings, producing an LLM-enhanced class representation:

$$\mathbf{v}_j^{\text{LLM}} = \text{Encoder}(T_j).$$

These embeddings replace the original class-name embeddings and are used as initial node features in the Graph Attention Network (GAT).

5 Experimental Results

5.1 Quantitative Results

These results highlight three key observations:

(1) **High-quality sentence embeddings + MLP** already provide strong linear separability.
(2) **Self-training consistently improves performance**, regardless of architecture.
(3) **LLM-based embeddings harm performance**, suggesting misalignment between generated class descriptions and review-level semantics.

5.2 Analysis of GNN-based Models

Although graph-based class encoders were evaluated using GAT architectures, they consistently underperformed compared to MLP-based classifiers with strong sentence embeddings. We attribute this behavior to two factors. First, the quality of document representations obtained from pretrained sentence transformers dominates performance, reducing the marginal benefit of explicit graph propagation. Second, the taxonomy graph contains many semantically similar sibling nodes, causing message passing to blur discriminative signals rather than refine them.

As a result, GNN-based smoothing tends to overshare information across neighboring classes, which can be detrimental in a weakly supervised setting where initial labels are noisy.

6 Case Study and Discussion

6.1 Successful Case

We first analyze a successful prediction example to understand when the proposed silver-label-based framework performs well.

In this case, the document contains explicit product-related terms that are semantically aligned with a small subset of taxonomy classes. During silver label generation, these classes appear consistently among the Top-K similarity candidates and exceed the similarity threshold τ. As a result, the document receives stable pseudo labels even before self-training.

The hierarchy-aware completion step further strengthens prediction consistency by propagating labels to ancestor classes. This leads to a prediction set that is both semantically accurate and hierarchically valid.

Notably, self-training reinforces these correct predictions by increasing confidence for already high-probability labels, indicating that the model benefits most when initial silver labels are reliable.

This example demonstrates that the proposed approach is particularly effective when semantic similarity between documents and class descriptions is strong and unambiguous.

6.2 Failure Case

We also observe failure cases where the model assigns incorrect or overly general labels.

A common failure scenario occurs when multiple sibling classes share highly overlapping keywords and semantic representations. In such cases, cosine similarity scores for these classes are very close, causing instability in Top-K selection.

Although the fallback strategy guarantees at least one label per document, it can introduce noise when the highest similarity score does not correspond to the true semantic category. Once an incorrect class is selected, the hierarchy-aware completion step propagates this error to ancestor classes, amplifying the impact of the initial mistake.

Self-training does not fully correct these errors, as the model may reinforce incorrect pseudo labels if they consistently exceed the confidence threshold γ. This highlights a limitation of confidence-based self-training when initial pseudo labels are biased.

6.3 Discussion

From these observations, we identify two key limitations of the proposed framework.

First, similarity-based silver label generation struggles with fine-grained distinctions among sibling classes that share similar semantics. Second, hierarchy-aware completion, while enforcing structural consistency, can propagate early-stage errors rather than correct them.

These limitations suggest that future work could benefit from incorporating more discriminative class representations or adaptive thresholding strategies that account for local taxonomy structure.

7 Implementation Details

- **Hyperparameters**: We used a batch size of 64, a dropout rate of 0.5 for the MLP, and a momentum coefficient of 0.999 for model updates.
- **Reproducibility**: All experiments were conducted with a fixed random seed (42). We provided a `run_all.sh` script to automate the pipeline from embedding generation to final prediction.

7.1 Prediction and Inference Procedure

This section describes the final prediction process used for Kaggle submission.

Given a trained model f_θ, each test document d_i is mapped to a probability vector

$$f_\theta(d_i) \in [0,1]^{|C|}.$$

Binary predictions are obtained by applying a fixed threshold γ to each class probability:

$$\hat{y}_{ij} = \mathbb{I}\big[f_\theta(d_i)_j \geq \gamma\big].$$

If no class satisfies the threshold, the class with the maximum predicted probability is selected as a fallback to ensure non-empty predictions.

Finally, hierarchy consistency is enforced by assigning all ancestor classes of each predicted label. The resulting binary label vector is used directly for Kaggle submission.

All LLM usage complied with the course policy, and the total number of API calls was kept below the allowed limit. Prompts and generated outputs are provided in the submitted repository for verification.

8 Conclusion

8.1 Summary of Findings

This project demonstrated a robust pipeline for unsupervised hierarchical text classification. We successfully combined semantic similarity, taxonomic constraints, and advanced deep learning techniques such as self-training and momentum ensembling. Our results indicate that while graph-based models offer structural advantages, simple MLP architectures paired with high-quality embeddings and iterative refinement remain highly effective for large-scale product classification.

8.2 Future Work

Future research could focus on hybrid GNN–Transformer architectures that more tightly integrate the attention mechanisms of the text encoder with the graph structure. Additionally, exploring dynamic momentum scaling could further optimize the balance between initial silver labels and pseudo-label discovery.